AUTOM-02: Settings API

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 2 of 9 | **Created:** January 2026 | **Last Updated:** 10/02/2026

The Settings API (also called Settings 2.0) is Dynatrace's modern REST API for configuration management. It provides a unified way to manage all Dynatrace settings through JSON objects with schema validation.

Table of Contents

1. Introduction
2. API Fundamentals
3. Working with Settings Objects
4. Common Operations
5. Best Practices
6. Next Steps

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Token	A platform token (or OAuth client) with `settings:objects:read` and `settings:objects:write` — or, on an environment that still has classic access tokens, an access token with `settings.read` and `settings.write` (see Authentication)
Tenant URL	Your Dynatrace SaaS tenant URL
HTTP client	curl, Postman, or similar tool

Learning Objectives

By the end of this notebook, you will:

- Understand the Settings 2.0 API architecture
- Know how to read and write configuration objects
- Be able to list available schemas and their structure
- Handle common API patterns and error cases

---

1. Introduction

Settings API vs Classic Config API

Aspect	Settings API (2.0)	Classic Config API
Format	Unified JSON schemas	Mixed endpoints
Validation	Schema-based	Varies by endpoint
New features	All new features	Legacy only
Recommended	Yes	Deprecated for new configs

Key Concepts

Concept	Description
Schema	Defines structure and validation rules for a config type
Object	An instance of configuration following a schema
Scope	Where the config applies (tenant, host, service, etc.)
Object ID	Unique identifier for a configuration object

Sprint 1.337 (April 2026) Updates

Extensions credentials gained `EXTENSION_AUTHENTICATION` enum. The Credentials API (both Environment and Config) now supports an `EXTENSION_AUTHENTICATION` scope value. This enables extension-based authentication scenarios — for example, granting a stored credential the ability to authenticate Extensions API calls without expanding its broader scope.

Extensions Action endpoint signature change. `PUT /extensions/{extensionName}/monitoringConfigurations/{configurationId}/actions` now returns `agIds` (plural array) and deprecates the singular `agId` and `agName` properties. Update any code parsing the response to handle the array form.

> **Tokens:** Plan Extensions automation around **platform tokens (`dt0s16`)** with `Authorization: Bearer …` rather than the classic `dt0c01` access token (`Authorization: Api-Token …`): classic access tokens don't exist once the environment is on Latest Dynatrace. (`dt0s01` is a different token — the account SCIM API token — not a platform token.) For the migration steps, see Upgrade from classic access tokens to platform tokens (DT docs).
>
> **Sources:**
>
> - Dynatrace API changelog 1.337 (DT docs) — `/credentials` *"Added enum value: EXTENSION_AUTHENTICATION"*; `/extensions` *"Added properties: agIds"*
> - Access tokens — token prefixes (DT docs) — *"dt0s16 Platform Token enabling programmatic access to Dynatrace platform services."*; *"dt0s01 This is an API token. It's used as an authorization method: a valid token allows the user to make changes within the Dynatrace account through SCIM."*
> - Upgrade from classic access tokens to platform tokens (DT docs) — *"Replace Authorization: Api-Token with Authorization: Bearer in every migrated integration."*

---

2. API Fundamentals

Base URL Pattern

```
https://{tenant-id}.live.dynatrace.com/api/v2/settings
```

Authentication

Two token types work with the Settings API, and which ones your environment has depends on where it is in the upgrade to Latest Dynatrace:

Token	Header	Scopes	Available in
Platform token (`dt0s16`) or OAuth client bearer token	`Authorization: Bearer <token>`	`settings:objects:read` , `settings:objects:write`	Hybrid and Latest environments
Classic access token (`dt0c01`)	`Authorization: Api-Token <token>`	`settings.read` , `settings.write`	Classic and hybrid environments only — not in Latest Dynatrace

Start new automation with a platform token: it is the only option once the environment reaches Latest Dynatrace. The examples in this notebook use the platform-token header; on an environment that still has classic access tokens, the same calls work with the `Api-Token` header and the classic scopes:

Platform token:

```
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Bearer {platform-token}"
```

Classic access token (classic and hybrid environments only):

```
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Api-Token {token}"
```

> **Sources:**
>
> - Settings API - List objects (DT docs) — *"Platform Token / OAuth: Required scope: settings:objects:read"*
> - Settings API - POST an object (DT docs) — *"Platform Token / OAuth: Required scope: settings:objects:write"*
> - Dynatrace API authentication (DT docs) — *"Platform tokens use the Bearer realm in the Authorization HTTP header."*
> - Upgrade from classic access tokens to platform tokens (DT docs) — *"Platform tokens are also available alongside classic access tokens."*
>
> (hybrid); *"Classic access tokens don't exist in latest environments, and v2/apiTokens isn't available."*

Core Endpoints

Endpoint	Method	Purpose
`/api/v2/settings/schemas`	GET	List available schemas
`/api/v2/settings/schemas/{schemaId}`	GET	Get schema definition
`/api/v2/settings/objects`	GET	List/search objects
`/api/v2/settings/objects`	POST	Create objects

Endpoint	Method	Purpose
`/api/v2/settings/objects/{objectId}`	GET	Get specific object
`/api/v2/settings/objects/{objectId}`	PUT	Update object
`/api/v2/settings/objects/{objectId}`	DELETE	Delete object

Listing Available Schemas

To see all available configuration types:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Bearer {platform-token}" \
  -H "Accept: application/json"
```

Settings 2.0 Schema Catalog by Domain

This is the repo's consolidated Settings 2.0 schema catalog. **Corrected 07/01/2026:** the SLO row previously read `builtin:slo` — the actual schemaId, confirmed at the `dynatrace_slo_v2` Terraform resource docs, is `builtin:monitoring.slo`. That wrong value had propagated into the SLO series' own REFERENCE.md and SLO-05 before being fixed the same day; this table is now the single source other series should cite.

Reading the Upgrade status column

A schema being **available today** and a schema **surviving your upgrade to the latest Dynatrace** are two different questions, and several rows below answer them differently.

Question	What it means	Where it is answered
Is it deprecated?	Dynatrace has named a successor. An end-of-life date may or may not be published, and the schema keeps working meanwhile.	Product documentation, release notes
Is it blocked at upgrade?	The schema stops answering once the tenant moves to the latest Dynatrace.	The ready-made *Check your upgrade readiness* dashboard

The `Upgrade status` values below are read from that dashboard, observed **07/31/2026**, and cross-checked **09/18/2026** against the list Dynatrace now publishes: Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs) — *"None of the schemas on this page are visible in Latest Dynatrace."* and *"If you use configuration-as-code or other automations that reference these schemas by ID, you need to update these automations before the schemas are removed from your environment."* The published page is the authoritative list; the readiness dashboard remains the tenant-specific check. Blocking is triggered by **your tenant's own upgrade** rather than a calendar date, so the timing is yours; the scope is not. Re-check against the dashboard in your tenant before planning around any single row.

This matters most for config-as-code. Monaco resolves a schema before writing objects (`--settings-schema`), and the Terraform provider maps its resources onto the same schemas — so a `Blocked` row takes its Monaco config type and its Terraform resource with it. `/api/v2/settings/objects` itself is unaffected; it is the individual schema that goes away, not the endpoint.

Domain	Schema ID	Configuration type	Upgrade status
Classic config	`builtin:management-zones`	Management zones (see MZ2POL for the policy-migration path)	**Blocked**
Classic config	`builtin:tags.auto-tagging`	Auto-tagging rules	**Blocked**

Domain	Schema ID	Configuration type	Upgrade status
Classic config	`builtin:tags.manual-tagging`	Manual tagging	**Blocked**
Classic config	`builtin:naming.hosts` / `.services` / `.processes-and-containers`	Conditional naming rules	**Blocked**
Alerting	`builtin:alerting.profile`	Alerting profiles	**Blocked** — successor is workflow-based notification (WFLOW, ALERT-03)
Alerting	`builtin:problem.notifications`	Problem notifications	**Blocked** — same successor
Alerting	`builtin:alerting.maintenance-window`	Maintenance windows	**Blocked** — successor is platform maintenance windows
Alerting	`builtin:anomaly-detection.infrastructure-hosts`	Host anomaly detection (there is no `builtin:anomaly-detection.hosts` schema)	Carries forward — its `.high-gc` sub-schema is on the removed list
Alerting	`builtin:anomaly-detection.services`	Service anomaly detection	Carries forward
Alerting	`builtin:anomaly-detection.metric-events`	Custom metric-event alerts (`dynatrace_metric_events` Terraform resource — not the singular, nonexistent `dynatrace_metric_event`)	**Blocked** — recreate as DQL-based detectors on `builtin:davis.anomaly-detectors`
Alerting	`builtin:anomaly-detection.infrastructure-disks`	Classic disk anomaly detection	**Blocked** — successor is the newer disk alerting
Alerting	`builtin:davis.anomaly-detectors`	DQL-based anomaly detectors	Carries forward — the Gen3 target for metric-event migrations
SLO	`builtin:monitoring.slo`	Service level objectives — **not** `builtin:slo`	**Blocked** — takes `dynatrace_slo_v2` and the Monaco path with it (SLO-05)
Service	`builtin:settings.calculated-service-metrics`	Calculated service metrics	**Blocked** — replaced by the new calculated metrics for services (removed-schemas list: "Replaced by calculated service metrics")
Logs	`builtin:logmonitoring.log-custom-attributes` / `.log-dpp-rules` / `.log-buckets-rules` / `.log-events`	Classic log processing, attributes, buckets and events	**Blocked** — successor is OpenPipeline (OPMIG); do not point new automation at these
Cloud	`builtin:cloud.aws`	Classic AWS connection	**Blocked** — recreate as a Cloud Observability connection (CLOUD-02)
Infrastructure	`builtin:os.services.monitoring`	Classic Windows services monitoring	**Blocked** — replaced by OS services monitoring

Domain	Schema ID	Configuration type	Upgrade status
Automation	`builtin:issue-tracking.integration`	Releases issue-tracking integrations	**Verify** — flagged by the readiness dashboard (observed 07/31/2026) but not on Dynatrace's published removed-schemas list (checked 10/02/2026). Confirm in your tenant's readiness dashboard before planning around it
Network	`builtin:networkzones.zones`	Network zones — replaces the deprecated `/api/v2/networkZones` Environment API endpoints (GET deprecated in 1.339, PUT/DELETE in 1.340); see M2S-03/04	**Verify** — the dashboard blocks `builtin:networkzones`, and prefix matching sweeps `.zones` in with it. The global toggle and the zone definitions are distinct schemas, so confirm which one is affected before acting
OpenPipeline	`builtin:openpipeline.<scope>.pipelines` / `.routing` / `.ingest-sources`	Per-data-type family, e.g. `builtin:openpipeline.logs.pipelines` — **not** a single generic schema; see OPIPE, OPMIG, SL2DT-03, NRLC-09	Carries forward
Service detection	`builtin:enhanced-endpoints-for-sdv1`	Service detection v1: auto-detect every endpoint and emit per-endpoint metrics (available from 1.330; on by default for environments created at 1.330+, and not configurable for those created at 1.333+; environments created at 1.329 or earlier get it at 1.333, off by default)	Carries forward — but must be **enabled** on every scope; scopes with it disabled break on upgrade

This table is not exhaustive of what is blocked. The readiness dashboard carries roughly 70 `builtin:` schema rules; the rows above are the ones this repo teaches or that readers commonly automate. Run the scan against your own tenant rather than treating an absence here as a clean bill of health. For the full migration sequence these rows sit inside, see the **Classic → Gen3 Platform** doorway in the `-START-HERE-` navigation playbook.

Not everything is a Settings 2.0 object. FINOPS has no row above because no Settings 2.0 schema exists for it — cost/budget configuration is driven by the Account Management API. **ONBRD** is split: OneAgent/ActiveGate *installer download* is driven by the Deployment API's installer endpoints, not a Settings object, but update policy, update windows and the default monitoring mode *are* Settings 2.0 schemas — `builtin:deployment.oneagent.updates`, `builtin:deployment.activegate.updates`, `builtin:deployment.management.update-windows` and `builtin:deployment.oneagent.default-mode`. For cost configuration and installers, go to the API directly (see FINOPS/ONBRD REFERENCE.md for the specific endpoints).

> **Sources:**
>
> - Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs) — no entry for `builtin:issue-tracking.integration` (checked 10/02/2026)
> - Dynatrace API changelog 1.339 (DT docs) — *"The /networkZones endpoints are deprecated. Use the Settings API endpoint /api/v2/settings/objects with schema builtin:networkzones.zones to list network zones."*; 1.340 (DT docs) deprecates `PUT` / `DELETE` `/networkZones/{id}`

> - Enhanced endpoints for SDv1 (DT docs) — feature-availability table by environment creation version
> - Deployment schemas (DT docs): OneAgent updates, ActiveGate updates, Update windows, OneAgent default mode

---

3. Working with Settings Objects

Object Structure

Every settings object follows this structure:

```
{
  "schemaId": "builtin:management-zones",
  "scope": "environment",
  "value": {
    "name": "Production",
    "rules": [...]
  }
}
```

Field	Description
`schemaId`	The schema this object follows
`scope`	Where the config applies
`value`	The actual configuration data

Scope Types

Scope	Format	Example
Environment (tenant)	`environment`	`environment`
Host	`HOST-{id}`	`HOST-ABC123DEF456`
Host group	`HOST_GROUP-{id}`	`HOST_GROUP-XYZ789`
Service	`SERVICE-{id}`	`SERVICE-DEF456ABC`
Application	`APPLICATION-{id}`	`APPLICATION-123ABC`

Reading Objects

List all objects for a schema:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?schemaIds=builtin:management-zones"
\
  -H "Authorization: Bearer {platform-token}"
```

Get a specific object:

```
curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
  -H "Authorization: Bearer {platform-token}"
```

Query Parameters

Parameter	Description	Example
`schemaIds`	Filter by schema(s)	`schemaIds=builtin:management-zones`
`scopes`	Filter by scope(s)	`scopes=HOST-ABC123`
`fields`	Select specific fields	`fields=objectId,value`
`pageSize`	Results per page	`pageSize=100`

---

4. Common Operations

> **On the two examples below.** Management zones and auto-tagging are used here because they are the clearest illustrations of the request shape — a named object with a rule list — and because most existing tenants already have them. Both schemas are marked **Blocked** in the catalog above: they work today and stop answering when the tenant upgrades to the latest Dynatrace. Read them as *API mechanics*, not as configuration to go and create. For the replacements, see MZ2POL (management zones → policies and segments) and FAQ entry 02 (tagging strategy).

Create a Management Zone

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?validateOnly=true" \
  -H "Authorization: Bearer {platform-token}" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:management-zones",
    "scope": "environment",
    "value": {
      "name": "Production-Web",
      "rules": [{
        "enabled": true,
        "type": "ME",
        "attributeRule": {
          "entityType": "SERVICE",
          "serviceToHostPropagation": false,
          "serviceToPGPropagation": false,
          "conditions": [{
            "key": "SERVICE_TAGS",
            "operator": "TAG_KEY_EQUALS",
            "tag": "environment"
          }]
        }
      }]
    }
  }]'
```

Create an Auto-Tagging Rule

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?validateOnly=true" \
  -H "Authorization: Bearer {platform-token}" \
  -H "Content-Type: application/json" \
  -d '[{
    "schemaId": "builtin:tags.auto-tagging",
    "scope": "environment",
    "value": {
      "name": "Application",
      "rules": [{
        "enabled": true,
        "type": "ME",
        "valueFormat": "{Service:DetectedName}",
        "valueNormalization": "Leave text as-is",
        "attributeRule": {
          "entityType": "SERVICE",
          "serviceToHostPropagation": false,
          "serviceToPGPropagation": false,
          "conditions": [{
            "key": "SERVICE_TAGS",
            "operator": "TAG_KEY_EQUALS",
            "tag": "environment"
          }]
        }
      }]
    }
  }]'
```

How the rule payload is shaped. Each rule has a `type` — `ME`, `DIMENSION` or `SELECTOR` for management zones; `ME` or `SELECTOR` for auto-tagging — and an `ME` rule carries an `attributeRule` with the `entityType`, one to 30 `conditions`, and the propagation flags that apply to that entity type (`serviceToHostPropagation` / `serviceToPGPropagation` for `SERVICE`). Which fields a condition needs depends on its `key` and `operator`: a `*_TAGS` key with `TAG_KEY_EQUALS` takes the tag key in `tag`; a name key such as `SERVICE_NAME` takes `stringValue` and `caseSensitive` instead. The classic Configuration API shape — `key: {type, attribute}` plus `comparisonInfo` — is **not** accepted by these schemas.

Validate first. Both requests above carry `validateOnly=true`, so they run the full server-side validation without saving anything. Check each item's `code` in the response; when it is `200`, drop the parameter to create the object. Before writing a payload for any other schema, fetch its definition (`GET /api/v2/settings/schemas/<schemaId>`) — the schema version in your tenant is authoritative, and some fields are required only for particular entity types or operators.

The request shape is identical for any schema that carries forward — swap the `schemaId` and the `value` block. `builtin:davis.anomaly-detectors` and the `builtin:openpipeline.<scope>.*` family are good next examples to try, and both survive the upgrade.

> **Sources:**
>
> - Management zones schema table (DT docs) — rule `type` *"The element has these enums ME DIMENSION SELECTOR"*; condition `tag` *"Format: [CONTEXT]tagKey:tagValue"*
> - Auto-tagging schema table (DT docs) — rule `type` *"The element has these enums ME SELECTOR"*
> - Schema definitions vendored in the Terraform provider (Dynatrace GitHub): `builtin:management-zones`, `builtin:tags.auto-tagging` — `conditions` `minObjects` 1 / `maxObjects` 30; per-field preconditions on `key`, `operator` and `entityType`; auto-tagging acceptance test uses `TAG_KEY_EQUALS` with a bare tag key, read 10/02/2026
> - Settings API key concepts (DT docs) — *"Setting validateOnly=true runs the full server-side validation of the submitted objects without persisting anything."*

Update an Object

```
curl -X PUT "https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
  -H "Authorization: Bearer {platform-token}" \
  -H "Content-Type: application/json" \
  -d '{
    "updateToken": "{updateToken-from-your-last-GET}",
    "value": {
      "name": "Production-Web-Updated",
      "rules": [...]
    }
  }'
```

`updateToken` is optional but worth sending: it is returned on every read, and with it the update (or delete) only proceeds if nobody changed the object since you fetched it. Without it the change is applied unconditionally. The PUT reference notes that the read scope is required as well as the write scope.

Delete an Object

```
curl -X DELETE "https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
  -H "Authorization: Bearer {platform-token}"
```

Bulk Operations

Create multiple objects in one request:

```
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects" \
  -H "Authorization: Bearer {platform-token}" \
  -H "Content-Type: application/json" \
  -d '[
    {"schemaId": "...", "scope": "...", "value": {...}},
    {"schemaId": "...", "scope": "...", "value": {...}},
    {"schemaId": "...", "scope": "...", "value": {...}}
  ]'
```

Each object in the batch is processed independently and gets its own status `code` in the response array — check every item rather than the top-level status. **A partially failed batch is not rolled back:** the items that succeeded stay created.

> **Sources:** Settings API key concepts (DT docs) — *"Including it on a subsequent update or delete means the operation only proceeds if the object hasn't changed since you last fetched it"*; *"A partially failed batch does not roll back successful items."* Settings API - PUT an object (DT docs) — *"The settings.read scope is required as well."*

Error Handling

Status codes for `POST /api/v2/settings/objects`:

Code	Meaning	Action
200	Success	Every object in the request succeeded
207	Multi-status	Objects in the batch got different results — check each item's `code`
400	Schema validation failed	Read the item's `constraintViolations`; fetch the schema definition
401	Unauthorized	Verify the token and the header realm (`Bearer` vs `Api-Token`)

Code	Meaning	Action
403	Forbidden	Check the token scopes and, for a platform token, the user's permissions
404	Not Found	Object or schema doesn't exist
409	Conflicting resource	Re-read the current object and reconcile before retrying
429	Too many requests	The environment's request queue is full — back off and retry

Validation Errors

The response is an **array** with one entry per submitted object, so errors are read per item:

```
[
  {
    "code": 400,
    "error": {
      "code": 400,
      "message": "Constraints violated",
      "constraintViolations": [
        {
          "path": "&lt;property path&gt;",
          "message": "...",
          "parameterLocation": "PAYLOAD_BODY",
          "location": "..."
        }
      ]
    }
  }
]
```

> **Sources:** Settings API - POST an object (DT docs) — *"207 Settings Object Response [] Multi-status: different objects in the payload resulted in different statuses."*; *"409 Settings Object Response [] Failed. Conflicting resource."*; response elements `code`, `error`, `constraintViolations`, `parameterLocation`. Access limits (DT docs) — *"When you reach the limit, your requests return the response code 429."*

---

5. Best Practices

API Token Security

Practice	Description
Least privilege	Only grant required scopes
Environment-specific	Separate tokens for dev/prod
Secret management	Use vaults, not hardcoded values
Rotation	Rotate tokens regularly

Rate Limiting

Limit Type	Recommendation
Request throttling	No fixed per-minute quota is documented. Each environment has a request thread pool with a queue; when both are full the API returns `429`
Retry strategy	Exponential backoff on 429 errors
Payload size	Keep each bulk `POST` under the 1 MB payload limit — split large batches
Batch operations	Use bulk endpoints where possible

Idempotency

Approach	Description
`externalId` on create	Give each object a stable, caller-defined `externalId`. A `POST` with an `externalId` that already exists on the target scope replaces that object instead of creating a duplicate — an upsert, safe to re-run
`updateToken` on update/delete	Send the token from your last read so a concurrent change is rejected rather than overwritten
Track object IDs	Store the returned `objectId` (or look objects up later with the `externalIds` parameter)

> **Sources:** Access limits (DT docs) — *"Every environment has a limited thread pool (with a queue) for request processing."*; *"The payload size is limited to 1 MB."* Settings API key concepts (DT docs) — *"if an object with the given externalId already exists on the target scope, the request replaces it rather than creating a duplicate."*

Schema Discovery Pattern

Before creating objects, understand the schema:

1. **List schemas** to find the right one
2. **Get schema definition** for field requirements
3. **Review existing objects** for examples
4. **Validate** before bulk operations — `POST /api/v2/settings/objects?validateOnly=true` runs full validation without saving

```
# Step 1: Find the schema
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas" \
  -H "Authorization: Bearer {platform-token}" | jq '.items[] | select(.schemaId | contains("management"))'

# Step 2: Get schema details
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/schemas/builtin:management-zones" \
  -H "Authorization: Bearer {platform-token}"

# Step 3: See existing examples
curl "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?schemaIds=builtin:management-zones" \
  -H "Authorization: Bearer {platform-token}"

# Step 4: Dry-run the payload (nothing is saved)
curl -X POST "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?validateOnly=true" \
  -H "Authorization: Bearer {platform-token}" \
  -H "Content-Type: application/json" \
  -d @payload.json
```

---

6. Next Steps

When to Graduate from Settings API

Consider moving to higher-level tools when:

Scenario	Recommended Tool
Managing many configurations	Monaco
Need version control	Monaco or Terraform
Part of larger IaC pipeline	Terraform
Building a custom application	SDK

Continue the Series

Next Notebook	Focus
AUTOM-03: Monaco	Configuration-as-code with YAML

Additional Resources

- Settings API Documentation
- Settings API key concepts
- Upgrade from classic access tokens to platform tokens
- API Token Management
- Schema Reference

Summary

In this notebook, you learned:

- The Settings 2.0 API architecture and endpoints
- How to work with schemas, scopes, and objects
- Common CRUD operations for configuration
- Best practices for API token security and rate limiting

> **Key Takeaway:** The Settings API is the foundation for all Dynatrace configuration automation. Understanding it helps you debug issues with higher-level tools like Monaco and Terraform.

---

*Continue to **AUTOM-03: Monaco** to learn configuration-as-code patterns.*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.